

Answers

KS_what_we_know_esp32

Referenced file: `KS_what_we_know_esp32.ino`

Task 1: Program Analysis

The program in `KS_what_we_know_esp32.ino`:

- Configures pin 35 as an INPUT_PULLUP pin using `pinMode(35, INPUT_PULLUP)`
- Initializes serial communication at 115200 baud with `Serial.begin(115200)`
- Continuously reads the state of pin 35 in a loop using `digitalRead(35)`
- Prints the raw value (0 or 1) to the serial console using `printf("%d\n", left)`

Output behavior:

- When the button is **not pressed**: prints `1` (HIGH, due to INPUT_PULLUP)
- When the button is **pressed**: prints `0` (LOW, button connects pin to GND)

Task 2: Function Explanations

a. `pinMode`

- Configures a specific pin to behave as either INPUT or OUTPUT
- In this program: `pinMode(35, INPUT_PULLUP)` sets pin 35 as input with internal pull-up resistor
- Pull-up means the pin reads HIGH (1) when not connected to ground

b. `digitalRead`

- Reads the digital value from a specified pin
- Returns either HIGH (1) or LOW (0)
- In this program: `digitalRead(35)` reads the current state of the button on pin 35

c. `Serial.begin`

- Initializes serial communication between the ESP32 and the computer
- The parameter (115200) is the baud rate, which must match on both ends
- Without this, no communication with Serial Monitor would be possible

d. `printf`

- Prints formatted text and values to the serial output
- `%d` is a format specifier for decimal (integer) values
- In this program: `printf("%d\n", left)` prints the value of `left` followed by a newline

Task 3: Special Pins

a. Pin 0

- On ESP32, GPIO 0 is a special pin used during boot
- It is connected to the on-board LED and affects boot mode
- Reading from pin 0 can give unexpected results if the board is in certain states
- Not recommended for general use as it can interfere with normal operation

b. Pin 35

- GPIO35 is an **input-only** pin on ESP32
- It cannot be used as an output pin
- It is safe to use for digital input, but cannot drive any external components
- In the context of `digitalRead`, it works like any other input pin

Task 4: Modified Program (PBL)

Modified file: `KS_what_we_know_esp32_solution.ino`

The program now uses an `if-else` control structure to print different messages based on the button state:

```
#include <Arduino.h>

void setup() {
  pinMode(35, INPUT_PULLUP);
  Serial.begin(115200);
}

void loop() {
  int left = digitalRead(35);

  if (left == LOW) {
    printf("Knap trykket!\n");
  } else {
    printf("Knap ikke trykket\n");
  }

  delay(100);
}
```

Explanation:

- `if (left == LOW)` checks if the button is pressed (pin pulled to GND)
- When true, prints "Knap trykket!"
- When false (button not pressed), prints "Knap ikke trykket"
- Added `delay(100)` to reduce serial output flooding

KS_conditional_esp32

Referenced files: `KS_what_we_know_esp32.ino`, `KS_conditional_esp32.ino`

Task 1: Program Comparison

Difference between the two programs:

KS_what_we_know_esp32.ino:

- Continuously prints the raw button state (0 or 1) to serial
- No conditional logic
- Outputs on every loop iteration

KS_conditional_esp32.ino:

- Uses a conditional (**if**) statement to only print when the button state changes
- Tracks previous state with **lprev** variable
- Only outputs when there is a transition (button pressed or released)

The condition:

```
if (left != lprev)
```

This condition checks if the current button state (**left**) is different from the previous state (**lprev**). When true, it means the button state has changed, and the code inside the block executes.

Task 2: Behavior of KS_conditional_esp32

KS_conditional_esp32 is more efficient because:

- It only prints to serial when the button state **changes** (transition from pressed to released or vice versa)
- It avoids flooding the serial monitor with repeated messages
- It reduces unnecessary processing and communication

Behavior:

- When button is pressed: prints once
- When button is released: prints once
- While button is held: no output (only on state change)

This is useful for detecting button press/release events rather than continuous state.

Task 3: Modified Program (PBL)

Modified file: KS_conditional_esp32_solution.ino

```
#include <Arduino.h>

int lprev = LOW;

void setup() {
  pinMode(35, INPUT_PULLUP);
  Serial.begin(115200);
}
```

```
void loop() {
  int left = digitalRead(35);

  if (left != lprev) {
    if (left == LOW) {
      printf("Knap trykket!\n");
    } else {
      printf("Knap sluppet!\n");
    }
    lprev = left;
  }

  delay(10);
}
```

Explanation:

- Outer `if (left != lprev)` detects state change
- Inner `if-else` determines whether it's a press or release
- `lprev = left;` updates the previous state for next iteration
- "Knap trykket!" prints when button transitions to pressed (LOW)
- "Knap sluppet!" prints when button transitions to released (HIGH)

p04_KS5

Referenced file: `p04_ks5.c`

Task 1-3: Complete Program

Solution file: `p04_ks5_solution.c`

```
#include <stdio.h>

int main() {
  char name[] = "Kelly";
  char gender = 'F';
  int age = 13;
  float bmi = 23.0;

  // Print name, age, and BMI
  printf("%s, age %d, has a bmi of %.1f", name, age, bmi);

  // Print gender text
  if (gender == 'F' || gender == 'f') {
    printf(", female, meaning she is ");
  } else if (gender == 'M' || gender == 'm') {
    printf(", male, meaning he is ");
  }

  // Print BMI category
```

```
if (bmi < 18.5) {
    printf("Underweight\n");
} else if (bmi >= 18.5 && bmi <= 24.9) {
    printf("Normal weight\n");
} else if (bmi >= 25.0 && bmi <= 29.9) {
    printf("Overweight\n");
} else {
    printf("Obese\n");
}

return 0;
}
```

Task 4: Expected Outputs

Test case 1: Kelly

Kelly, age 13, has a bmi of 23.0, female, meaning she is Normal weight

Test case 2: Mogens

Mogens, age 45, has a bmi of 31.0, male, meaning he is Obese

p04_KS6

Referenced file: [p04_ks6.c](#)

Task 1-3: Complete Program with BMI Calculation

Solution file: [p04_ks6_solution.c](#)

```
#include <stdio.h>

int main() {
    char name[] = "Kelly";
    char gender = 'F';
    int age = 13;
    float h = 2.0;    // height in meters
    float w = 30.0;   // weight in kg
    float bmi = w / (h * h);

    // Print name, age, and calculated BMI
    printf("%s, age %d, has a bmi of %.1f", name, age, bmi);

    // Print gender text
    if (gender == 'F' || gender == 'f') {
```

```
        printf(", female, meaning she is ");
    } else if (gender == 'M' || gender == 'm') {
        printf(", male, meaning he is ");
    }

    // Print BMI category based on calculated value
    if (bmi < 18.5) {
        printf("Underweight\n");
    } else if (bmi >= 18.5 && bmi <= 24.9) {
        printf("Normal weight\n");
    } else if (bmi >= 25.0 && bmi <= 29.9) {
        printf("Overweight\n");
    } else {
        printf("Obese\n");
    }

    return 0;
}
```

Task 4: Expected Outputs

Test case 1: Kelly

- BMI calculation: $30 / (2.0 * 2.0) = 30 / 4 = 7.5$

Kelly, age 13, has a bmi of 7.5, female, meaning she is Underweight

Test case 2: Mogens

- BMI calculation: $500 / (2.0 * 2.0) = 500 / 4 = 125.0$

Mogens, age 45, has a bmi of 125.0, male, meaning he is Obese

p04_KS7

Referenced file: [p04_ks7.ino](#)

Task 1-4: Complete Counter Program

Solution file: [p04_ks7_solution.ino](#)

```
#include <Arduino.h>

// Global counter variable
int c = 0;
```

```
// Button pins
const int leftButtonPin = 35; // Example pin for left button
const int rightButtonPin = 34; // Example pin for right button

void setup() {
  // Configure buttons as inputs with pull-up resistors
  pinMode(leftButtonPin, INPUT_PULLUP);
  pinMode(rightButtonPin, INPUT_PULLUP);

  // Start serial communication
  Serial.begin(115200);
}

void loop() {
  // Read button states
  int leftButtonState = digitalRead(leftButtonPin);
  int rightButtonState = digitalRead(rightButtonPin);

  // Check left button (decrement)
  if (leftButtonState == LOW) {
    c--;
  }

  // Check right button (increment)
  if (rightButtonState == LOW) {
    c++;
  }

  // Send counter value over serial
  Serial.print("Counter: ");
  Serial.println(c);

  // Wait 100 milliseconds
  delay(100);
}
```

Additional Notes

Pin Configuration:

- The solution assumes left button on pin 35 and right button on pin 34
- Both configured as INPUT_PULLUP, so LOW = pressed, HIGH = not pressed

Behavior:

- Counter decrements when left button is pressed
- Counter increments when right button is pressed
- Current value sent to Serial Monitor every 100ms
- Can be observed in both Serial Monitor and Serial Plotter

Potential Improvements:

- Could add debounce logic to prevent multiple counts from a single press
 - Could use `if (leftButtonState == LOW && leftButtonPrev == HIGH)` pattern to detect only the press moment
 - Could track previous button states to only count on transitions
-

File References

For all questions, the corresponding answer files are:

- `KS_what_we_know_esp32_solution.ino`
- `KS_conditional_esp32_solution.ino`
- `p04_ks5_solution.c`
- `p04_ks6_solution.c`
- `p04_ks7_solution.ino`

Each solution file implements the required functionality while maintaining the original question's intent and constraints.